

INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO



Programas Clase 27

Unidad de Aprendizaje: *Sistemas Distribuidos*

Profesor: *Coronilla Contreras Ukranio*

Alumnos:

De la Rosa Gutiérrez Kaleb Yael

Grupo*: 7CM2*

Evidencia

Item 1

```
import java.io.IOException;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.time.Duration;

/**
 * Ejercicio 1 (Item 1) - Práctica Cloud Translation API:
 *
 * Consumo de una API REST pública sin token de validación: JokeAPI.
 * Utiliza el cliente HTTP síncrono nativo de Java (HttpClient) para enviar
una
 * solicitud GET y obtener un chiste de la categoría deseada.
 *
 * Materia: Sistemas Distribuidos - ESCOM IPN
 * Alumno : Kaleb Yael De La Rosa Gutiérrez
 * Grupo  : 7CM1
 */
public class ClienteJokeAPI {

    // Endpoint público oficial de JokeAPI v2 (categoría Programming)
    private static final String JOKE_API_URL =
"https://v2.jokeapi.dev/joke/Programming?format=json";

    public static void main(String[] args) {
        String endpoint = JOKE_API_URL;
        if (args.length > 0) {
            endpoint = args[0];
        }


System.out.println("============================================================
======================");
        System.out.println("   EJERCICIO 1: CONSUMO DE API REST PUBLICA SIN
AUTENTICACION (JokeAPI)         ");
        System.out.println("   Alumno: Kaleb Yael De La Rosa Gutiérrez |
Grupo: 7CM1                        ");
```

```
System.out.println("============================================================
======================");
        System.out.println(">>> Preparando cliente HTTP síncrono...");
        System.out.println(">>> URL Destino: " + endpoint);

        // 1. Construcción del cliente HTTP síncrono
        HttpClient client = HttpClient.newBuilder()
                .version(HttpClient.Version.HTTP_1_1)
                .connectTimeout(Duration.ofSeconds(10))
                .followRedirects(HttpClient.Redirect.NORMAL)
                .build();

        // 2. Construcción de la solicitud HTTP GET
        HttpRequest request = HttpRequest.newBuilder()
                .GET()
                .uri(URI.create(endpoint))
                .header("Accept", "application/json")
                .header("User-Agent", "Java11-HttpClient-ESCOM/1.0")
                .build();

        try {
            System.out.println("\n>>> Enviando solicitud HTTP GET a
JokeAPI...");
            long tiempoInicio = System.currentTimeMillis();

            // 3. Envío síncrono y bloqueo hasta recibir respuesta
            HttpResponse<String> response = client.send(request,
HttpResponse.BodyHandlers.ofString());
            long latencia = System.currentTimeMillis() - tiempoInicio;

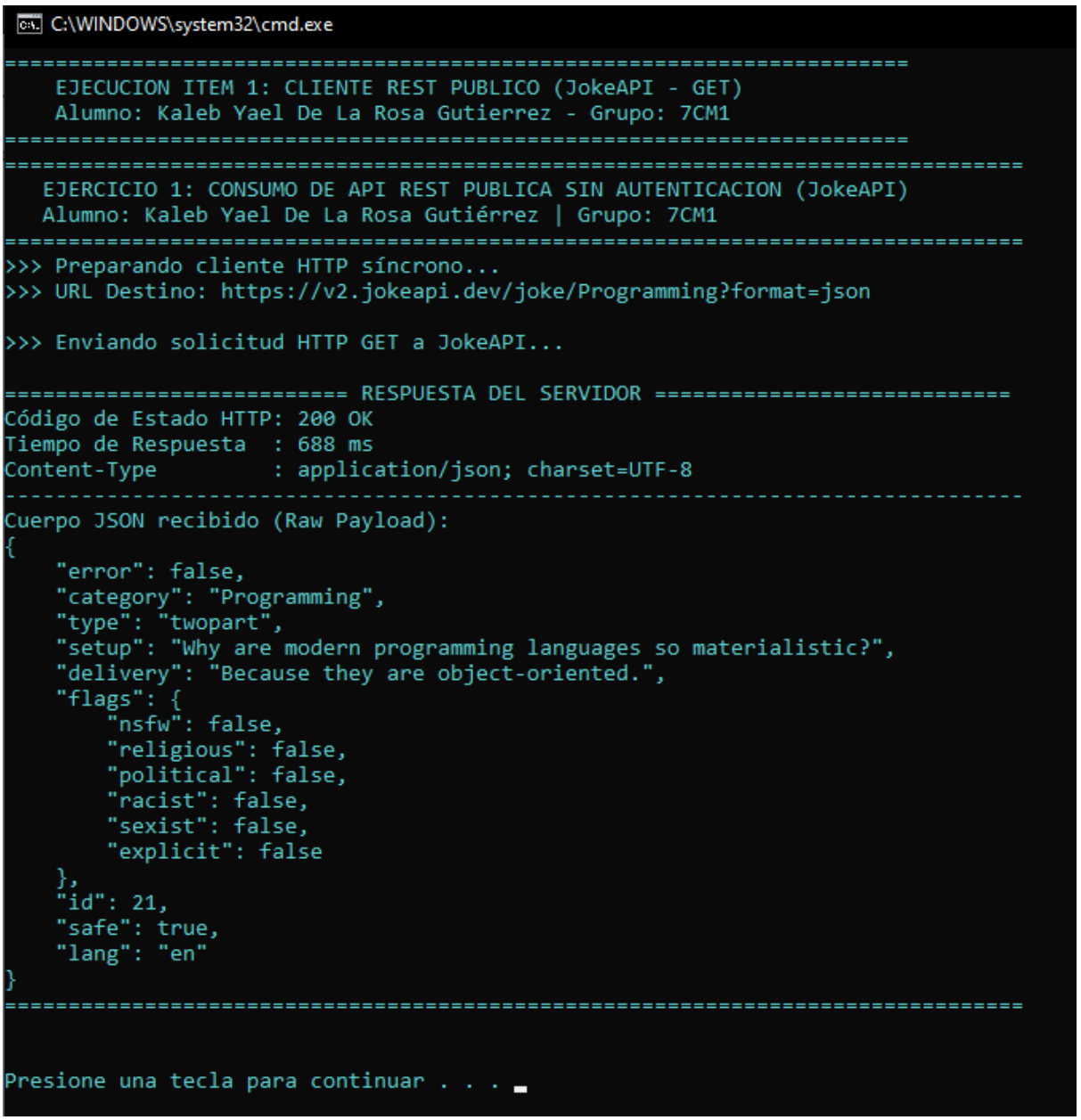
            // 4. Despliegue de resultados
            System.out.println("\n========================== RESPUESTA DEL
SERVIDOR ============================");
            System.out.println("Código de Estado HTTP: " +
response.statusCode() + " OK");
            System.out.println("Tiempo de Respuesta  : " + latencia + " ms");
            System.out.println("Content-Type         : " +
response.headers().firstValue("Content-Type").orElse("N/A"));

System.out.println("------------------------------------------------------------
----------------------");
```

```
            System.out.println("Cuerpo JSON recibido (Raw Payload):");
            System.out.println(response.body());

System.out.println("============================================================
======================\n");

        } catch (IOException | InterruptedException e) {
            System.err.println("[ERROR] Ocurrió una excepción al comunicarse
con la API: " + e.getMessage());
            e.printStackTrace();
        }
    }
}
```



Item 2

```java
import java.io.IOException;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.time.Duration;

/**
 * Ejercicio 2 (Item 2) - Práctica Cloud Translation API:
```

```java
 *
 * Creación de un recurso ficticio mediante solicitud HTTP POST a
JSONPlaceholder.
 * Envía un payload en formato JSON con la cabecera 'Content-Type:
application/json; charset=UTF-8',
 * conteniendo el Grupo, Nombre del Alumno y userId.
 *
 * Materia: Sistemas Distribuidos - ESCOM IPN
 * Alumno : Kaleb Yael De La Rosa Gutiérrez
 * Grupo  : 7CM1
 */
public class ClientePlaceholder {

    private static final String POSTS_ENDPOINT =
"https://jsonplaceholder.typicode.com/posts";

    public static void main(String[] args) {
        String grupo = "7CM1";
        String alumno = "Kaleb Yael De La Rosa Gutiérrez";
        int userId = 1;

        if (args.length >= 2) {
            grupo = args[0];
            alumno = args[1];
        }


System.out.println("============================================================
======================");
        System.out.println("   EJERCICIO 2: CREACION DE RECURSO CON SOLICITUD
HTTP POST (JSONPlaceholder)   ");
        System.out.println("   Alumno: " + alumno + " | Grupo: " + grupo);

System.out.println("============================================================
======================");
        System.out.println(">>> Endpoint Destino: " + POSTS_ENDPOINT);

        // 1. Construcción del cuerpo JSON requerido por la práctica
        String jsonPayload = String.format(
                "{\n  \"title\": \"%s\",\n  \"body\": \"%s\",\n  \"userId\":
%d\n}",
                grupo, alumno, userId
```

```
        );

        System.out.println("\n>>> Cuerpo del mensaje HTTP (JSON) a enviar:");
        System.out.println(jsonPayload);

        // 2. Construcción del cliente HTTP síncrono
        HttpClient client = HttpClient.newBuilder()
                .version(HttpClient.Version.HTTP_1_1)
                .connectTimeout(Duration.ofSeconds(10))
                .build();

        // 3. Configuración de la solicitud POST con cabecera Content-Type
recomendada
        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create(POSTS_ENDPOINT))
                .header("Content-Type", "application/json; charset=UTF-8")
                .header("Accept", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(jsonPayload))
                .build();

        try {
            System.out.println("\n>>> Enviando solicitud HTTP POST...");
            long inicio = System.currentTimeMillis();

            // 4. Envío síncrono
            HttpResponse<String> response = client.send(request,
HttpResponse.BodyHandlers.ofString());
            long latencia = System.currentTimeMillis() - inicio;

            // 5. Presentación de resultados y verificación de ID generado
            System.out.println("\n============================ RESPUESTA DEL
SERVIDOR ============================");
            System.out.println("Código de Estado HTTP : " +
response.statusCode() + " (" + (response.statusCode() == 201 ? "201 Created -
ÉXITO" : "Respuesta recibida") + ")");
            System.out.println("Tiempo de Respuesta   : " + latencia + " ms");
            System.out.println("Encabezado Content-Type: " +
response.headers().firstValue("Content-Type").orElse("N/A"));

System.out.println("------------------------------------------------------------
----------------------");
```

```
            System.out.println("Respuesta JSON obtenida (con ID autogenerado
por el backend):");
            System.out.println(response.body());

System.out.println("============================================================
======================\n");

        } catch (IOException | InterruptedException e) {
            System.err.println("[ERROR] Error durante el envío de la solicitud
POST: " + e.getMessage());
            e.printStackTrace();
        }
    }
}
```

```
=========================================================================
  EJECUCION ITEM 2: CREACION DE RECURSO REST (JSONPlaceholder - POST)
  Alumno: Kaleb Yael De La Rosa Gutierrez - Grupo: 7CM1
=========================================================================
=============================================================================
   EJERCICIO 2: CREACION DE RECURSO CON SOLICITUD HTTP POST (JSONPlaceholder)
   Alumno: Kaleb Yael De La Rosa Gutierrez | Grupo: 7CM1
=============================================================================
>>> Endpoint Destino: https://jsonplaceholder.typicode.com/posts

>>> Cuerpo del mensaje HTTP (JSON) a enviar:
{
  "title": "7CM1",
  "body": "Kaleb Yael De La Rosa Gutierrez",
  "userId": 1
}

>>> Enviando solicitud HTTP POST...

============================ RESPUESTA DEL SERVIDOR ============================
Código de Estado HTTP : 201 (201 Created - ÉXITO)
Tiempo de Respuesta   : 652 ms
Encabezado Content-Type: application/json; charset=utf-8
-----------------------------------------------------------------------------
Respuesta JSON obtenida (con ID autogenerado por el backend):
{
  "title": "7CM1",
  "body": "Kaleb Yael De La Rosa Gutierrez",
  "userId": 1,
  "id": 101
}
=============================================================================


Presione una tecla para continuar . . .
```

Item 3

```java
import java.io.BufferedReader;
import java.io.File;
import java.io.FileReader;
import java.io.IOException;
import java.net.URI;
import java.net.URLEncoder;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.nio.charset.StandardCharsets;
import java.time.Duration;
import java.util.Scanner;

/**
 * Ejercicio 3 (Item 3) - Práctica Cloud Translation API:
 *
 * Consumo de la API de traducción de Google Cloud Platform (Cloud Translation API v2)
 * mediante el método POST con parámetros en la URL (Query Parameters):
 * - target=es (idioma destino: español)
 * - key=API_KEY (credencial de acceso a GCP)
 * - q=texto_a_traducir (codificado en URL con UTF-8)
 *
 * Texto de prueba especificado en la práctica:
 * "Political correctness is the attempt to control language and conduct by imposing
 * an ideological hegemony, which is the very antithesis of free speech."
 *
 * Materia: Sistemas Distribuidos - ESCOM IPN
 * Alumno : Kaleb Yael De La Rosa Gutiérrez
 * Grupo  : 7CM1
 */
public class ClienteTranslationAPI {

    private static final String BASE_URL = "https://translation.googleapis.com/language/translate/v2";

    // Texto exacto proporcionado en la práctica
    public static final String TEXTO_ORIGINAL =
```

```
            "Political correctness is the attempt to control language and
conduct by imposing an ideological hegemony, which is the very antithesis of
free speech.";

    public static void main(String[] args) {

System.out.println("============================================================
======================");
        System.out.println("   EJERCICIO 3: CONSUMO DE GOOGLE CLOUD
TRANSLATION API (POST CON API KEY)      ");
        System.out.println("   Alumno: Kaleb Yael De La Rosa Gutiérrez |
Grupo: 7CM1                         ");

System.out.println("============================================================
======================");

        // 1. Obtención de la API Key (por argumento, por archivo api_key.txt
o entrada estándar)
        String apiKey = obtenerApiKey(args);

        if (apiKey == null || apiKey.trim().isEmpty() ||
apiKey.equalsIgnoreCase("TU_API_KEY_AQUI")) {
            System.err.println("\n[ERROR] No se proporcionó una API Key válida
de Google Cloud.");
            System.err.println("Uso:");
            System.err.println("  1. Por argumento: java ClienteTranslationAPI
<TU_API_KEY>");
            System.err.println("  2. O crea un archivo 'api_key.txt' con tu
clave dentro de esta carpeta.");
            return;
        }

        apiKey = apiKey.trim();

        // 2. Texto a traducir y codificación en formato URL
(percent-encoding)
        String texto = TEXTO_ORIGINAL;
        if (args.length >= 2) {
            texto = args[1];
        }
```

```
        String textoCodificado = URLEncoder.encode(texto,
StandardCharsets.UTF_8);

        // 3. Construcción de la URL con Query Parameters según especificación
del manual de GCP
        String urlCompleta = String.format("%s?target=es&key=%s&q=%s",
BASE_URL, apiKey, textoCodificado);

        // Ocultar parte de la clave para despliegue seguro en pantalla
        String claveOculta = apiKey.length() > 8
                ? apiKey.substring(0, 4) + "..." +
apiKey.substring(apiKey.length() - 4)
                : "****";

        System.out.println(">>> Texto Original en Inglés:\n    \"" + texto +
"\"");
        System.out.println("\n>>> Parámetros de la Solicitud:");
        System.out.println("    - Endpoint Base: " + BASE_URL);
        System.out.println("    - target       : es (Español)");
        System.out.println("    - key          : " + claveOculta);
        System.out.println("    - Método HTTP  : POST");

        // 4. Construcción del cliente HTTP síncrono
        HttpClient client = HttpClient.newBuilder()
                .version(HttpClient.Version.HTTP_1_1)
                .connectTimeout(Duration.ofSeconds(15))
                .build();

        // 5. Creación de la solicitud POST (según la documentación de Google
REST v2, POST no requiere body si los parámetros van en URL)
        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create(urlCompleta))
                .header("Accept", "application/json")
                .header("User-Agent", "Java-Translation-Client-ESCOM/1.0")
                .POST(HttpRequest.BodyPublishers.noBody())
                .build();

        try {
            System.out.println("\n>>> Enviando solicitud HTTP POST a Google
Cloud Translation API...");
            long inicio = System.currentTimeMillis();
```

```
            HttpResponse<String> response = client.send(request,
HttpResponse.BodyHandlers.ofString());
            long latencia = System.currentTimeMillis() - inicio;

            System.out.println("\n========================== RESPUESTA DEL
SERVIDOR ============================");
            System.out.println("Código de Estado HTTP: " +
response.statusCode() + " " + (response.statusCode() == 200 ? "OK" : ""));
            System.out.println("Tiempo de Respuesta  : " + latencia + " ms");
            System.out.println("Content-Type         : " +
response.headers().firstValue("Content-Type").orElse("N/A"));

System.out.println("------------------------------------------------------------
----------------------");
            System.out.println("Respuesta JSON recibida de Google Cloud
Translation API:");
            System.out.println(response.body());

System.out.println("============================================================
======================\n");

            if (response.statusCode() == 200) {
                System.out.println(">>> [EXITO] Traducción completada
satisfactoriamente.");
            } else {
                System.out.println(">>> [AVISO] Revise si la clave de API
tiene habilitada y restringida la 'Cloud Translation API'.");
            }

        } catch (IOException | InterruptedException e) {
            System.err.println("[ERROR] Excepción durante la llamada HTTP a la
API: " + e.getMessage());
            e.printStackTrace();
        }
    }

    /**
     * Intenta resolver la API Key desde:
     * 1. Argumentos de línea de comandos (args[0])
     * 2. Archivo local api_key.txt
     * 3. Variable de entorno GCP_API_KEY
     * 4. Lectura interactiva por teclado
```

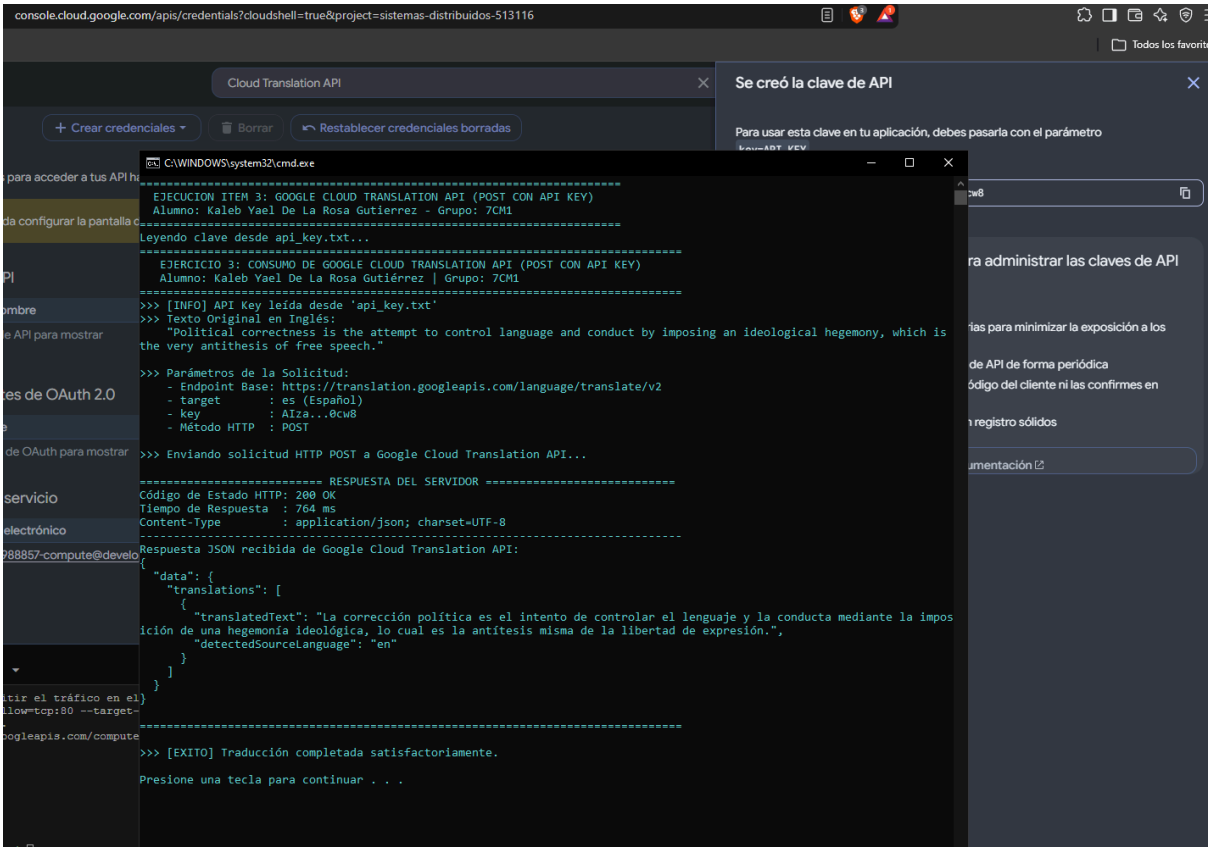
```
     */
    private static String obtenerApiKey(String[] args) {
        if (args.length > 0 && !args[0].trim().isEmpty()) {
            return args[0].trim();
        }

        File archivoKey = new File("api_key.txt");
        if (archivoKey.exists()) {
            try (BufferedReader br = new BufferedReader(new
FileReader(archivoKey))) {
                String linea = br.readLine();
                if (linea != null && !linea.trim().isEmpty()) {
                    System.out.println(">>> [INFO] API Key leída desde
'api_key.txt'");
                    return linea.trim();
                }
            } catch (IOException ignored) {}
        }

        String envKey = System.getenv("GCP_API_KEY");
        if (envKey != null && !envKey.trim().isEmpty()) {
            System.out.println(">>> [INFO] API Key leída desde variable de
entorno GCP_API_KEY");
            return envKey.trim();
        }

        System.out.println("\n>>> Ingrese su API Key de Google Cloud
Translation (o presione ENTER para salir): ");
        Scanner scanner = new Scanner(System.in);
        if (scanner.hasNextLine()) {
            return scanner.nextLine().trim();
        }

        return null;
    }
}
```


console.cloud.google.com/apis/credentials?cloudshell=true&project=sistemas-distribuidos-513116
Cloud Translation API
+ Crear credenciales
Borrar
Restablecer credenciales borradas
Se creó la clave de API
Para usar esta clave en tu aplicación, debes pasarla con el parámetro
C:\WINDOWS\system32\cmd.exe
EJECUCION ITEM 3: GOOGLE CLOUD TRANSLATION API (POST CON API KEY)
Alumno: Kaleb Yael De La Rosa Gutierrez - Grupo: 7CM1
Leyendo clave desde api_key.txt...
EJERCICIO 3: CONSUMO DE GOOGLE CLOUD TRANSLATION API (POST CON API KEY)
Alumno: Kaleb Yael De La Rosa Gutiérrez | Grupo: 7CM1
>>> [INFO] API Key leída desde 'api_key.txt'
>>> Texto Original en Inglés:
"Political correctness is the attempt to control language and conduct by imposing an ideological hegemony, which is the very antithesis of free speech."
>>> Parámetros de la Solicitud:
- Endpoint Base: https://translation.googleapis.com/language/translate/v2
- target : es (Español)
- key : AIza...0cw8
- Método HTTP : POST
>>> Enviando solicitud HTTP POST a Google Cloud Translation API...
RESPUESTA DEL SERVIDOR
Código de Estado HTTP: 200 OK
Tiempo de Respuesta : 764 ms
Content-Type : application/json; charset=UTF-8
Respuesta JSON recibida de Google Cloud Translation API:
{
"data": {
"translations": [
{
"translatedText": "La corrección política es el intento de controlar el lenguaje y la conducta mediante la imposición de una hegemonía ideológica, lo cual es la antítesis misma de la libertad de expresión.",
"detectedSourceLanguage": "en"
}
]
}
}
>>> [EXITO] Traducción completada satisfactoriamente.
Presione una tecla para continuar . . .